CORRIGÉ · PROBLÈME · ANNALES SPÉ

Mots de Dyck et algorithme de Heap

Énoncé

`let rec dyck = let f (x,y) = "a" ^ x ^ "b" ^ y in Cons ("", fun () -> map f (product dyck dyck))` ; D est l'ensemble des mots de ce flot.

1. Donner une grammaire non contextuelle G avec $L(G) = D$.
2. Énoncer une propriété de G qui assure que `dyck` n'énumère jamais deux fois le même mot, et la démontrer.

Algorithme de Heap : Heap(k, t) : si $k = 1$, afficher t ; sinon, pour $i = 0$ à $k-1$: exécuter Heap($k-1$, t), puis si k est pair échanger $t[i]$ et $t[k-1]$, sinon échanger $t[0]$ et $t[k-1]$.

1. Nombre $A(k)$ d'affichages lors de Heap(k, t).
2. Montrer : après Heap(k, t), si k est impair, t est inchangé ; si k est pair, les k premières cases ont subi une permutation circulaire vers la droite.
3. Montrer que Heap(k, t) affiche chaque permutation des k premières cases (les autres inchangées).
4. Écrire `pivot (k:int) (i:int) (t:'a array) : unit` (lignes 6 à 9).
5. Écrire `permstream_of_array (t:'a array) : 'a array stream`, flot fini des permutations dans l'ordre de l'algorithme de Heap, économe en mémoire.

Décoder l'énoncé

Ce que l'énoncé te donne.

- `dyck` = le mot vide, puis tous les $a\,x\,b\,y$ avec x, y dans `dyck`.

Ce qu'on te demande. Décrire le langage par une grammaire non ambiguë, puis prouver et programmer l'algorithme de Heap de façon paresseuse.

Corrigé

Question 30

Étape 1. Q30. Une grammaire pour D

$$G : \qquad S \to aSbS \mid \varepsilon.$$

Justification : le flot `dyck` contient le mot vide, puis les mots `"a" ^ x ^ "b" ^ y` pour tous les couples (x, y) de mots du flot (`product` les énumère tous). D est donc le plus petit langage contenant ε et stable par $(x, y) \mapsto axby$: c'est exactement $L(G)$ (récurrence sur la longueur des mots, ou sur la hauteur des arbres de dérivation).

Question 31

Étape 1. Q31. Pas de doublon : G est non ambiguë

Propriété : la grammaire G est **non ambiguë** (chaque mot a un unique arbre de dérivation). Comme `dyck` produit chaque arbre une seule fois (le produit énumère chaque couple une seule fois et f correspond à une règle), elle n'énumère jamais deux fois le même mot.

Lemme : dans un mot $w \in D$, tout préfixe a au moins autant de a que de b, et w a autant de a que de b (par récurrence sur la longueur de la dérivation : c'est vrai pour ε ; pour $axby$, un préfixe de ax a strictement plus de a, axb est équilibré, puis on ajoute un préfixe de y).

Unicité, par récurrence forte sur la longueur. Le mot vide n'a que la dérivation $S \to \varepsilon$. Soit $w \neq \varepsilon$; toute dérivation commence par $S \to aSbS$, et $w = axby$ avec $x, y \in D$. Le préfixe axb est le **plus court préfixe non vide** de w ayant autant de a que de b : d'après le lemme appliqué à x, les préfixes ax' avec x' préfixe de x ont un a de plus que de b. Ce plus court préfixe ne dépend que de w : la découpe (x, y) est donc unique, et par hypothèse de récurrence x et y (plus courts) ont une seule dérivation. Donc w aussi.

Question 32

Étape 1. Q32. Nombre d'affichages

Heap(1) affiche une fois : $A(1) = 1$. Pour $k \geqslant 2$, la boucle fait k appels à Heap$(k-1)$, et les échanges n'affichent rien : $A(k) = kA(k-1)$.

Récurrence : si $A(k-1) = (k-1)!$, alors $A(k) = k\,(k-1)! = k!$.

$$A(k) = k!.$$

Question 33

Étape 1. Q33. Effet de Heap(k) sur le tableau

Initialisation : Heap(1) affiche sans modifier : 1 est impair, t est inchangé.

Hérédité : soit $k \geqslant 2$, avec $P(k-1)$ vraie. Notons $a_0, \dots, a_{k-1}$ les k premières cases au départ.

Cas k pair : $k-1$ est impair, donc chaque appel Heap$(k-1)$ laisse t inchangé. Il ne reste que les échanges de $t[i]$ et $t[k-1]$ pour $i = 0, \dots, k-1$:

- $i = 0$: $[a_{k-1}, a_1, \dots, a_{k-2}, a_0]$;
- $i = 1$: $[a_{k-1}, a_0, a_2, \dots, a_{k-2}, a_1]$;
- après l'étape i : $[a_{k-1}, a_0, \dots, a_{i-1}, a_{i+1}, \dots, a_{k-2}, a_i]$;
- $i = k-2$: $[a_{k-1}, a_0, \dots, a_{k-3}, a_{k-2}]$; $i = k-1$: échange de $t[k-1]$ avec lui-même.

Résultat : $[a_{k-1}, a_0, a_1, \dots, a_{k-2}]$, la rotation circulaire vers la droite annoncée.

Cas k impair ($k \geqslant 3$) : $k-1$ est pair, donc chaque appel fait tourner vers la droite les $k-1$ premières cases. Un tour de boucle (appel puis échange de $t[0]$ et $t[k-1]$) transforme $[c_0, \dots, c_{k-1}]$ en :

$$[c_0, \dots, c_{k-2}, c_{k-1}] \xrightarrow{\text{rotation des } k-1} [c_{k-2}, c_0, \dots, c_{k-3}, c_{k-1}] \xrightarrow{t[0] \leftrightarrow t[k-1]} [c_{k-1}, c_0, \dots, c_{k-3}, c_{k-2}],$$

c'est-à-dire une rotation vers la droite des k cases. La boucle fait k tours, donc k rotations de k cases : on revient au tableau de départ. Les cases au-delà de k ne sont jamais touchées.

$P(k)$ est vraie, ce qui achève la récurrence.

Question 34

Étape 1. Q34. Toutes les permutations sont affichées

Montrons par récurrence : « Heap(k, t) affiche une fois chacune des $k!$ permutations des k premières cases, les autres étant inchangées ».

$k = 1$: une seule permutation, affichée une fois.

Hérédité : au tour i, l'appel Heap$(k-1)$ laisse $t[k-1]$ fixe (il ne touche que les $k-1$ premières cases) et affiche, par hypothèse de récurrence, toutes les permutations des $k-1$ premières cases avec cette valeur fixe en position $k-1$. Il suffit de voir que les k valeurs présentes en position $k-1$ au début des k tours sont **distinctes** :

- k pair : d'après le calcul de Q33, avant le tour $i \geqslant 1$ la dernière case contient a_{i-1}, et avant le tour 0 elle contient a_{k-1} : on obtient $a_{k-1}, a_0, a_1, \dots, a_{k-2}$, toutes distinctes ;
- k impair : chaque tour fait une rotation des k cases, donc avant le tour i la dernière case contient a_{k-1-i} : $a_{k-1}, a_{k-2}, \dots, a_0$, toutes distinctes.

Les k tours affichent donc des permutations deux à deux différentes (leur dernière case diffère d'un tour à l'autre), $(k-1)!$ par tour : $k!$ permutations distinctes, c'est-à-dire toutes (Q32).

Question 35

Étape 1. Q35. pivot

```
let pivot (k : int) (i : int) (t : 'a array) : unit =
  let swap a b = let x = t.(a) in t.(a) <- t.(b); t.(b) <- x in
  if k mod 2 = 0 then swap i (k - 1)     (* k pair : t[i] <-> t[k-1] *)
  else swap 0 (k - 1)                    (* k impair : t[0] <-> t[k-1] *)
```

Question 36

Étape 1. Q36. Le flot des permutations

Idée : on simule la récursion de Heap de façon paresseuse. `heap k next` renvoie le flot des tableaux affichés par Heap(k, t), suivi de `next ()`. Un affichage devient un `Cons` contenant une **copie** du tableau de travail ; les échanges ont lieu au moment où l'on demande la suite.

```
let permstream_of_array (t0 : 'a array) : 'a array stream =
  let t = Array.copy t0 in                (* tableau de travail *)
  let n = Array.length t in
  let rec heap k (next : unit -> 'a array stream) : 'a array stream =
    if k <= 1 then Cons (Array.copy t, next)          (* « afficher » *)
    else
      let rec loop i () =                             (* boucle pour i = 0 .. k-1 *)
        if i = k then next ()
        else heap (k - 1) (fun () -> pivot k i t; loop (i + 1) ())
      in
      loop 0 ()
  in
  heap n (fun () -> Nil)
```

Mémoire : un seul tableau de travail et une chaîne de fermetures de profondeur au plus n : $O(n)$ en plus des tableaux effectivement demandés, au lieu de stocker les $n!$ permutations. On n'en calcule une que si on la demande.

Attention : le flot fait des effets de bord quand on avance ; il doit être parcouru une seule fois (appeler deux fois la même queue rejouerait les échanges).

> **Le piège**
>
> En Q33, oublier l'échange du dernier tour ($i = k-1$) : dans la variante étudiée, il y a un échange après chaque appel, y compris le dernier.

La méthode

Dérécursifier paresseusement : passer à chaque appel une fonction « suite » qui calcule le reste du flot, et ne faire les effets qu'au moment où l'on force la queue.

Rappels de cours utilisés

- **Grammaire non ambiguë.** Une grammaire est non ambiguë si chaque mot du langage a un seul arbre de dérivation. Pour $S \to aSbS \mid \varepsilon$, le b qui ferme le premier a est déterminé : c'est la fin du plus court préfixe non vide ayant autant de a que de b.